

Databases 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 6 – Indexes

Index motivation

Students (SID: *int*, first_name: *string*, last_name: *string*)

```
SELECT *  
FROM Students  
WHERE SID = 12;
```

Simple scan: $O(N)$
Binary search: $O(\log_2(N))$
 N – number of disk blocks

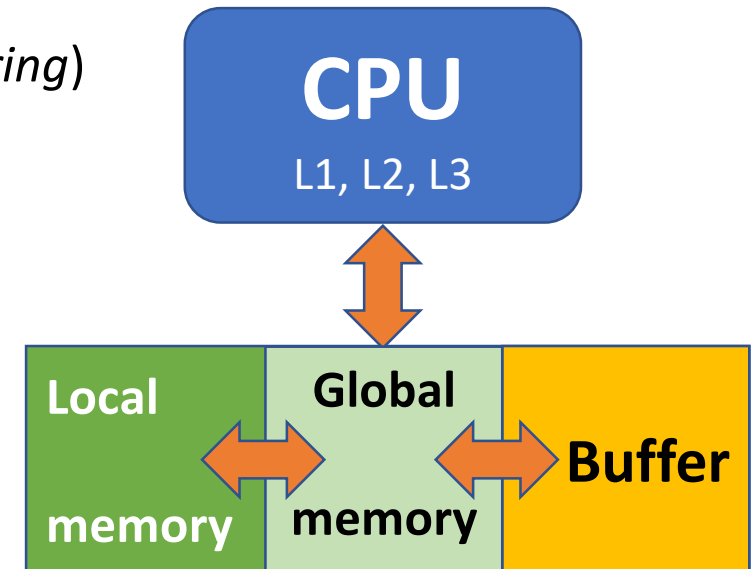
Records on disk are ordered
by primary key – SID

1 disk block = 4–32KB

Disk block 5465
Disk block 5466
Disk block 5467
Disk block 2233

1	Honey	Bunny	2	Ed	Sullivan	...	...	...	...
...	...	12	Mia	Wallace	...	...	...	...	...
...	...	...	...	34	Vincent	Vega	...	...	...
...	55	Butch	Coolidge	...	...	...	...	...	...

RAM



Index motivation

Students (SID: *int*, first_name: *string*, last_name: *string*)

```
SELECT *  
FROM Students  
WHERE first_name = 'Mia';
```

Not a primary key

Simple scan: $O(N)$
Binary search: ~~$O(\log_2(N))$~~
 N – number of disk blocks

Disk block 5465
Disk block 5466
Disk block 5467
Disk block 2233

1	Honey	Bunny	2	Ed	Sullivan	...	...	...	...
...	...	12	Mia	Wallace	...	...	...	...	...
...	...	...	...	34	Vincent	Vega	...	...	...
...	55	Butch	Coolidge	...	...	...	...	...	...

Indexes

- The index is a sorted list of keywords and next to each keyword is a set of page numbers pointing to the pages where each keyword can be found.
- An index is an on-disk or in-memory structure associated with a table or view that speeds retrieval of rows from the table or view.
- Well-designed indexes can reduce disk I/O operations and consume fewer system resources therefore improving query performance.
- Indexes can be helpful for a variety of queries that contain: SELECT, INSERT, UPDATE, DELETE.

Clustered and unclustered index

- **Clustering index** is when the ordering of data records on disk is the same as the ordering of data entries in the index.
- There can be only **one clustered index per table**, because the data rows themselves can be stored in only one order.
- Indexes are automatically created for **PRIMARY KEY** and **UNIQUE** constraints.
- For an attribute with **UNIQUE** constraint, DBMS automatically creates a **nonclustered index**.
- For a **PRIMARY KEY** DBMS automatically creates a **clustered index**.

Indexes in SQL

```
CREATE INDEX index_name  
ON table_name  
    (col1 [ASC | DESC], col2 [ASC | DESC],...);
```

```
DROP INDEX index_name  
ON table_name;
```

```
ALTER TABLE table_name  
    DROP INDEX index_name,  
    ADD index_name (col1, col2,...);
```

Indexes in SQL

```
Product(name, maker, category, price)
```

- By default a CLUSTERED index on PRIMARY KEY is created by DBMS at CREATE TABLE:

```
CREATE INDEX sd43fvg564dfg4  
ON Product (name);
```

- User defined indexes (**secondary indexes**):

```
CREATE INDEX idxProdMakCat  
ON Product (maker, category);
```

```
CREATE INDEX idxProdMakDCat  
ON Product (maker DESC, category);
```

```
CREATE INDEX idxProdCatMak  
ON Product (category, maker);
```



These are three different indexes!

Index on Multiple Attributes

If an index is created on attributes $\langle A_1, A_2, A_3, \dots, A_n \rangle$, the search key values are tuples with n values: $\langle v_1, v_2, v_3, \dots, v_n \rangle$.

- $\langle v_1, v_2, v_3 \rangle$

1. $\langle 5, 1, 34 \rangle$

2. $\langle 6, 0, 6 \rangle$

3. $\langle 7, 1, 9 \rangle$

4. $\langle 7, 2, 3 \rangle$

5. $\langle 7, 3, 2 \rangle$

6. $\langle 7, 3, 4 \rangle$

7. $\langle 7, 5, 1 \rangle$

Firstly, values are ordered by v_1 , if v_1 has equal values then they are ordered by v_2 , finally when values of v_2 are equal then for them v_3 values are ordered.

Function-based Indexes

```
Product(name, maker, category, price)
```

```
CREATE INDEX idxProdMakUPr1 ON Product (UPPER(maker), 0.01 * price);
```

```
SELECT maker, COUNT(*)  
FROM Product  
WHERE UPPER(maker) = 'ACME' AND 0.01 * price > 1000;
```

Indexes usage

```
Product(name, maker, category, price)
```

```
CREATE INDEX idxProdMakCat ON Product (maker, category);
```

```
SELECT maker, COUNT(*)  
FROM Product  
WHERE maker = 'ACME';
```

Yes

```
SELECT maker, category, COUNT(*), AVG(price)  
FROM Product  
WHERE maker = 'ACME' AND category = 'Electronics';
```

Yes

```
SELECT category, COUNT(*), AVG(price)  
FROM Product  
WHERE category = 'Electronics';
```

No

Indexes usage

```
Product(name, maker, category, price)
```

```
CREATE INDEX idxProdMakCat ON Product (maker, category);
```

```
CREATE INDEX idxProdMak ON Product (maker);
```

```
SELECT maker, COUNT(*)  
FROM Product  
WHERE maker = 'ACME';
```

Executed 1000x per minute

```
SELECT maker, category, COUNT(*), AVG(price)  
FROM Product  
WHERE maker = 'ACME' AND category = 'Electronics';
```

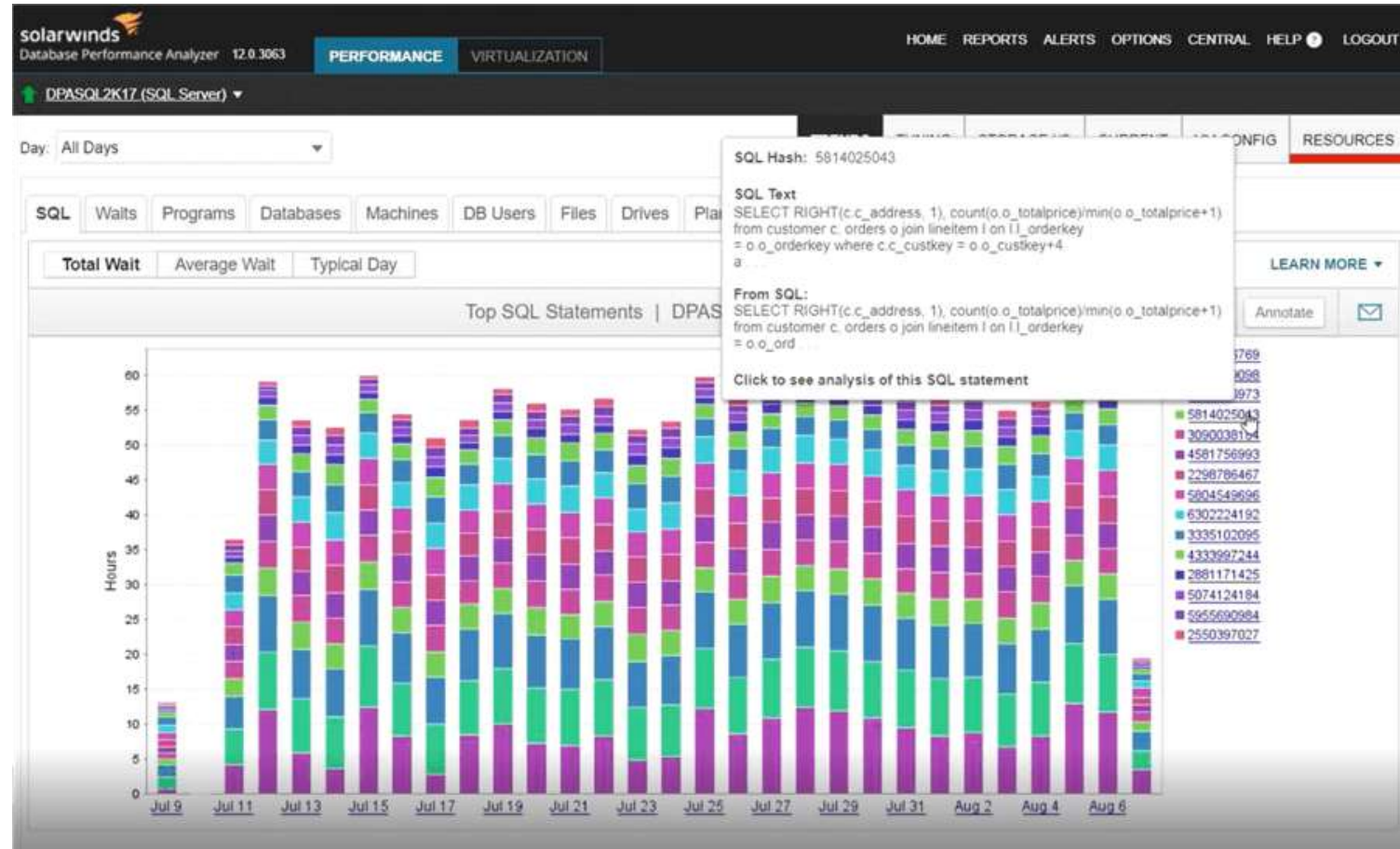
Which index to create?

- None
- One of them
- All of them

Executed 1000x per minute

Executed 10x per day

Database performance analysis



Automatic indexes – Oracle since version 19c

The automatic indexing process runs in the background every 15 minutes:

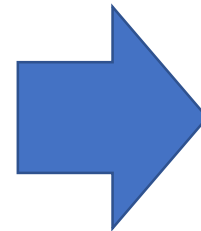
1. Automatic index candidates are identified based on the usage of table columns in SQL statements. Ensure that table statistics are up to date. Tables without statistics are not considered for automatic indexing. Tables with stale statistics are not considered for automatic indexing.
2. Index candidates are initially created as invisible and unusable. Invisible automatic indexes cannot be used by SQL statements in the application workload.
3. A sample of workload SQL statements is tested against the candidate indexes. During this verification phase, some or all candidate indexes will be built and made valid to measure the performance effect on SQL statements. If the performance of SQL statements is not improved using candidate indexes, they remain invisible.
4. Candidate valid indexes found to improve SQL performance will be made visible and available to the application workload. Candidate indexes that do not improve SQL performance will revert to invisible and be unusable after a short delay. During the verification stage, if an index is beneficial, but an individual SQL statement suffers a performance regression, a SQL plan baseline is created to prevent the regression when the index is made visible.
5. The automatic indexing deletes unusable and unused valid indexes process.

Covering indexes

- Covering indexes are a nonclustered index that resolves one or several similar query results directly with no access to its base table (on a disk), and without incurring in lookups.
- Covering indexes can improve query performance because all the data needed to meet the requirements of the query exists within the index itself.
- No data pages of the table or clustered index, are required to retrieve the requested data; therefore, reducing overall disk I/O.

```
Product(name, maker, category, price)
```

```
CREATE INDEX idxProdMakPr  
ON Product (maker, price);
```

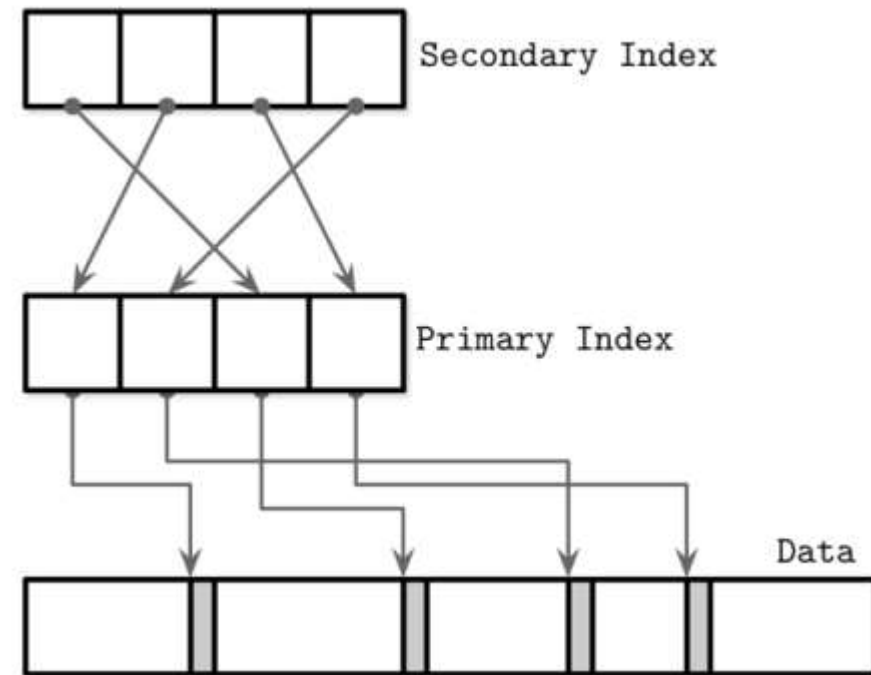
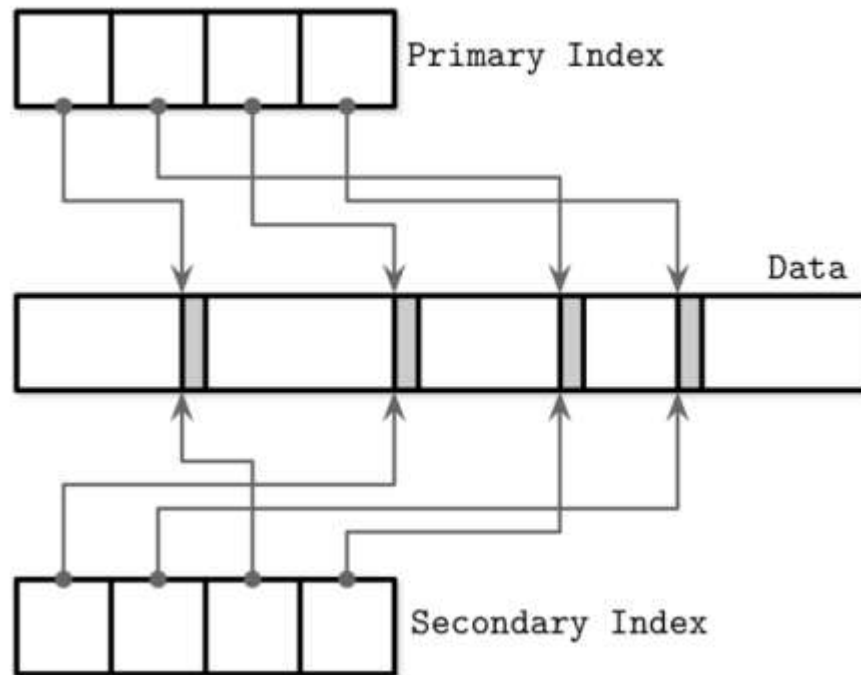


```
SELECT maker, AVG(price)  
FROM Product  
GROUP BY maker  
ORDER BY 2 DESC;
```

Primary and secondary indexes

Primary index – an index on a set of fields that includes the primary key.

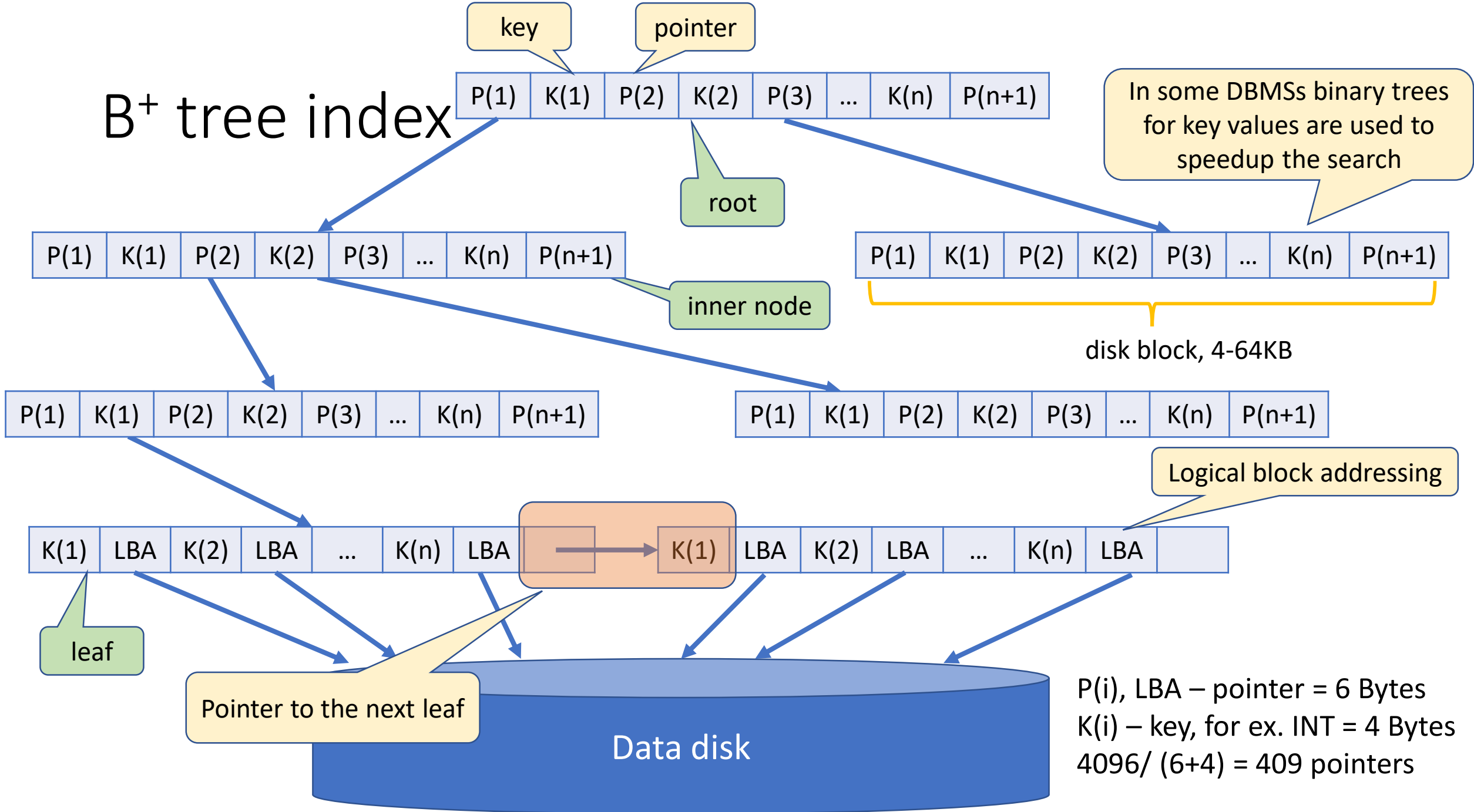
Secondary indexes – other indexes.



Types of indexes

- B⁺ tree index
 - A tree-like data structure is used. The data entries are arranged in sorted order by search key value, and a hierarchical search data structure is maintained that directs searches to the correct page of data entries.
- Hash index
 - A hash function (for ex. $\text{mod}()$ or $(a * \text{value} + b)$) maps search-key values to buckets, effectively partitioning the records of a data file into many small groups (the buckets). Buckets are represented by a block and possible overflow blocks.
 - Cannot support range searches.
- Bitmap index
 - Are specialized indices designed for easy querying on multiple keys. A bitmap is an array of bits.
 - Particularly applicable on fields with a small number of unique values.
- R tree index – efficient for spatial data (not covered)
- LSM tree (Log-Structured Merge Tree) – efficient for high number of writes (not covered)

B⁺ tree index



$P(i), LBA$ – pointer = 6 Bytes
 $K(i)$ – key, for ex. INT = 4 Bytes
 $4096 / (6+4) = 409$ pointers

Data types storage requirements in MySQL

Data Type	Storage Required
TINYINT	1 byte
SMALLINT	2 bytes
MEDIUMINT	3 bytes
INTEGER	4 bytes
BIGINT	8 bytes
FLOAT	4 bytes
DOUBLE	8 bytes
TIME	3 bytes
DATETIME	8 bytes
TIMESTAMP	4 bytes
CHAR(M)	$M * (1-3)$ bytes

P(i), LBA – pointer = 6 Bytes

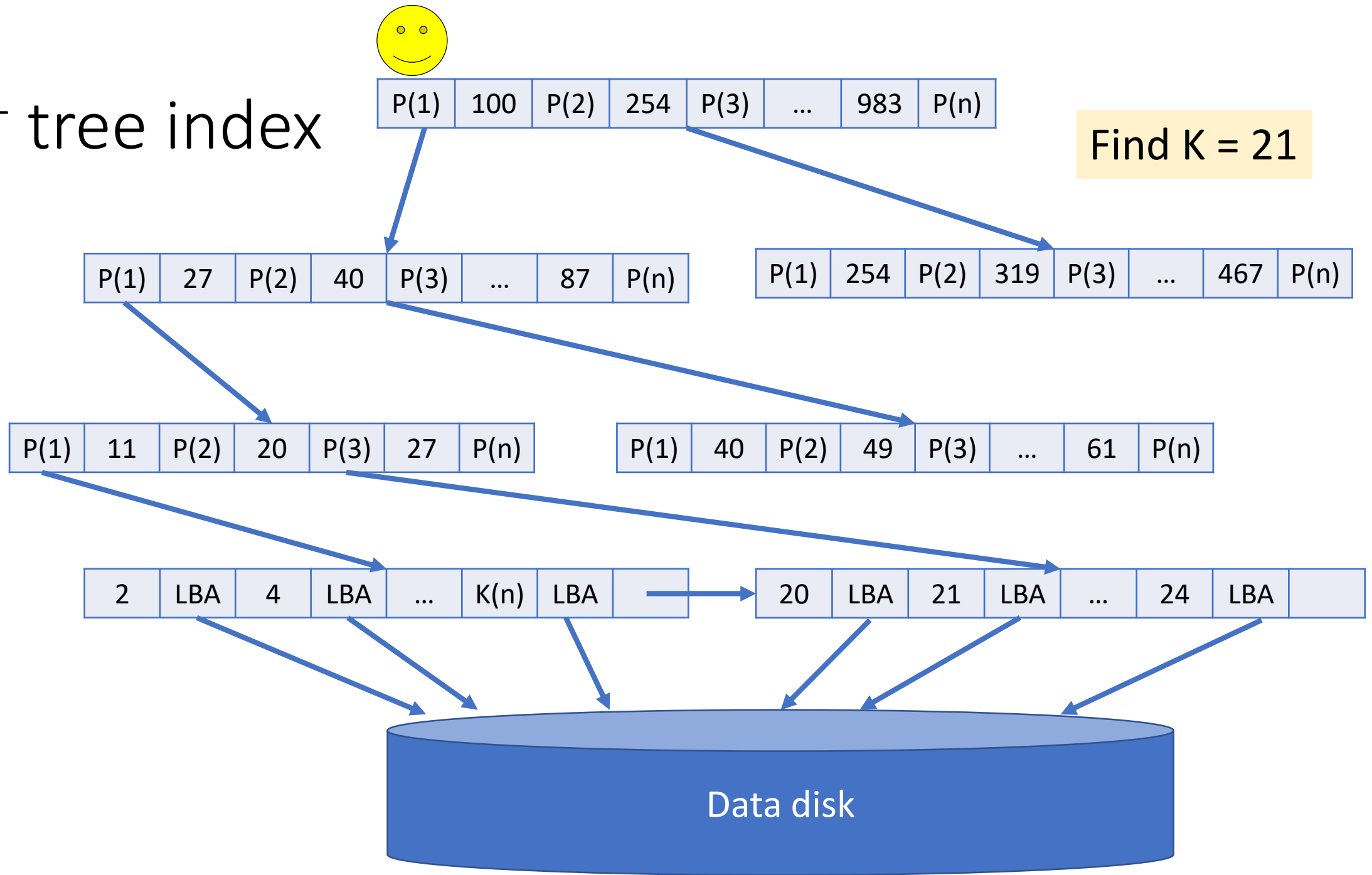
DATETIME = 8 Bytes

$4096 / (6+8) = 292$ pointers

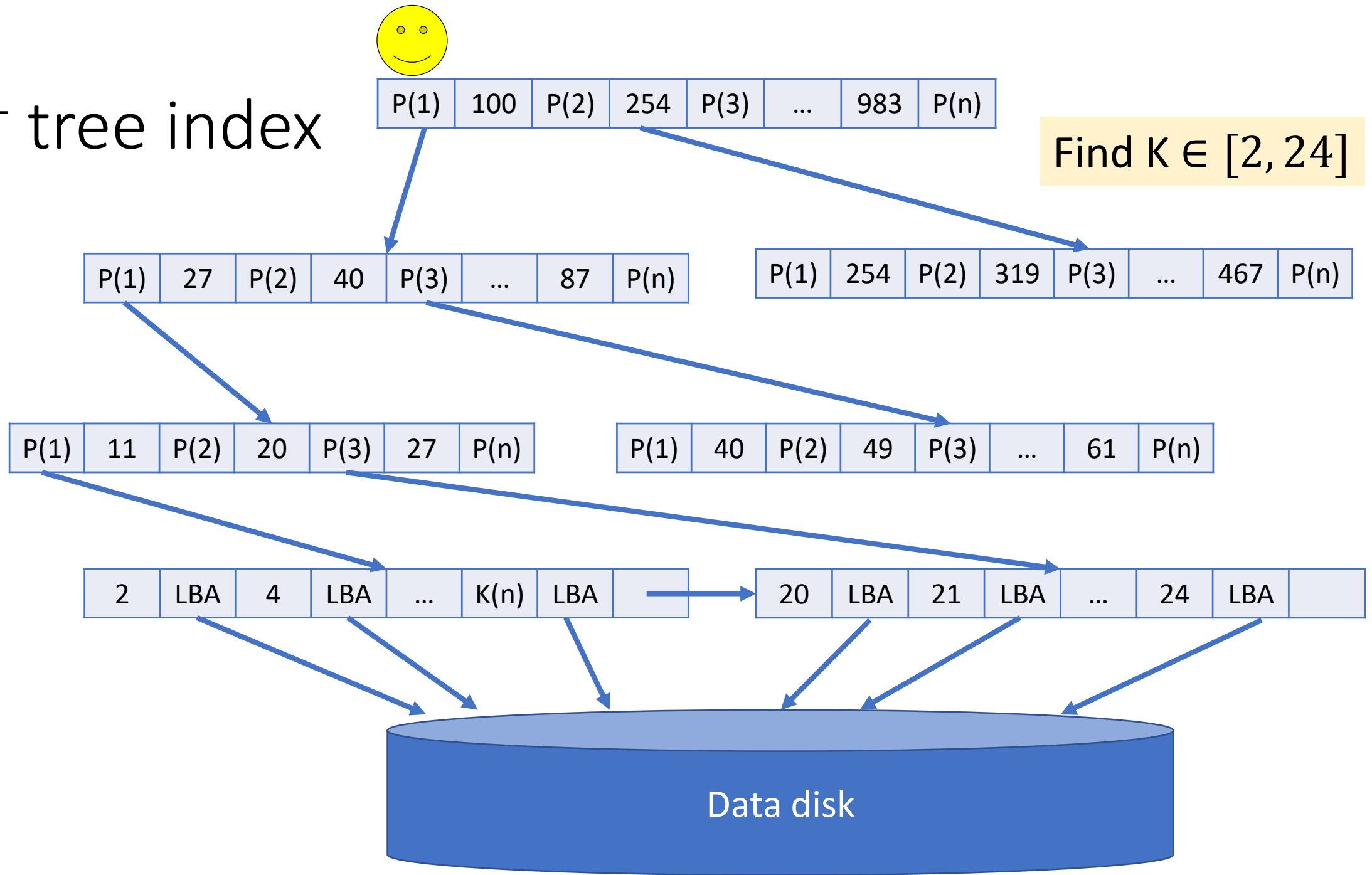
CHAR(10) = $10 * 2 = 20$ Bytes

$4096 / (6+20) = 157$ pointers

B⁺ tree index



B⁺ tree index



B⁺ tree index

- B⁺ tree index structure is the most widely used
- B⁺ tree index takes the form of a balanced tree in which every path from the root of the tree to a leaf of the tree is of the same length.
- B⁺ tree structure imposes performance overhead on insertion and deletion and adds space overhead. The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided.
- Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B⁺ tree structure.

B⁺ tree index

- A B⁺-tree node typically has **100 to 200 children**,
- It depends on the memory requirements of the keys and memory addresses.
- **Fill-factor** is the percent of available slots in the B+ Tree that are filled; is usually < 1 to leave slack for (quicker) insertions
- The typical fill-factor of the nodes is **67%**.
- Some DBMS enforces the minimum number of children, for ex. 50%
- The **fanout** is the number of pointers to child nodes coming out of a node
- The typical fanout is **133**.

B⁺ tree index

- A B⁺ tree of height 3 can index a table of $133^3 = 2,352,637$ records.
- In a B⁺ tree of height 4 that has an average fanout of 133 can be used to index a table of $133^4 = 312,900,700$ records.
- Top levels of tree sit *in the buffer pool*:
 - Level 1 = 1 page = 8 Kbytes
 - Level 2 = 133 pages = 1 Mbyte
 - Level 3 = 17,689 pages = 133 MBytes

B⁺ tree index cost model for search

- Let:
 - f = fanout (let's assume 200)
 - N = the total number of *pages* we need to index
 - F = fill-factor (usually $\approx 2/3$)
- Our B+ Tree needs to have room to index N/F pages!
 - We have the fill factor in order to leave some open slots for faster insertions
- What height (h) does our B+ Tree need to be?
 - $h=1 \rightarrow$ Just the root node- room to index f pages
 - $h=2 \rightarrow$ f leaf nodes- room to index f^2 pages
 - $h=3 \rightarrow$ f^2 leaf nodes- room to index f^3 pages
 - ...
 - $h \rightarrow f^{h-1}$ leaf nodes- room to index f^h pages!

We need a B⁺ Tree of height

$$h = \left\lceil \log_f \frac{N}{F} \right\rceil$$

B⁺ tree index cost model for search

- Note that if we have ***B*** available buffer pages, by the same logic:
 - We can store ***L_B*** levels of the B+ Tree in memory
 - where ***L_B*** *is the number of levels such that the sum of all the levels' nodes fit in the buffer:*
 - $B \geq 1 + f + \dots + f^{L_B-1} = \sum_{l=0}^{L_B-1} f^l$
- In summary: to do exact search:
 - We read in one page per level of the tree
 - Levels that we can fit in buffer are free (without I/O)!
 - Finally, we read in the actual record

$$\text{IO Cost: } \left\lceil \log_f \frac{N}{F} \right\rceil - L_B + 1$$

$$\text{where } B \geq \sum_{l=0}^{L_B-1} f^l$$

B⁺ tree index cost model for search

- To do range search, we just follow the horizontal pointers
- The IO cost is that of loading additional leaf nodes we need to access + the IO cost of loading each *page* of the results- we phrase this as “Cost(OUT)”

$$\text{IO Cost: } \left\lceil \log_f \frac{N}{F} \right\rceil - L_B + \text{Cost}(\text{OUT})$$

$$\text{where } B \geq \sum_{l=0}^{L_B-1} f^l$$

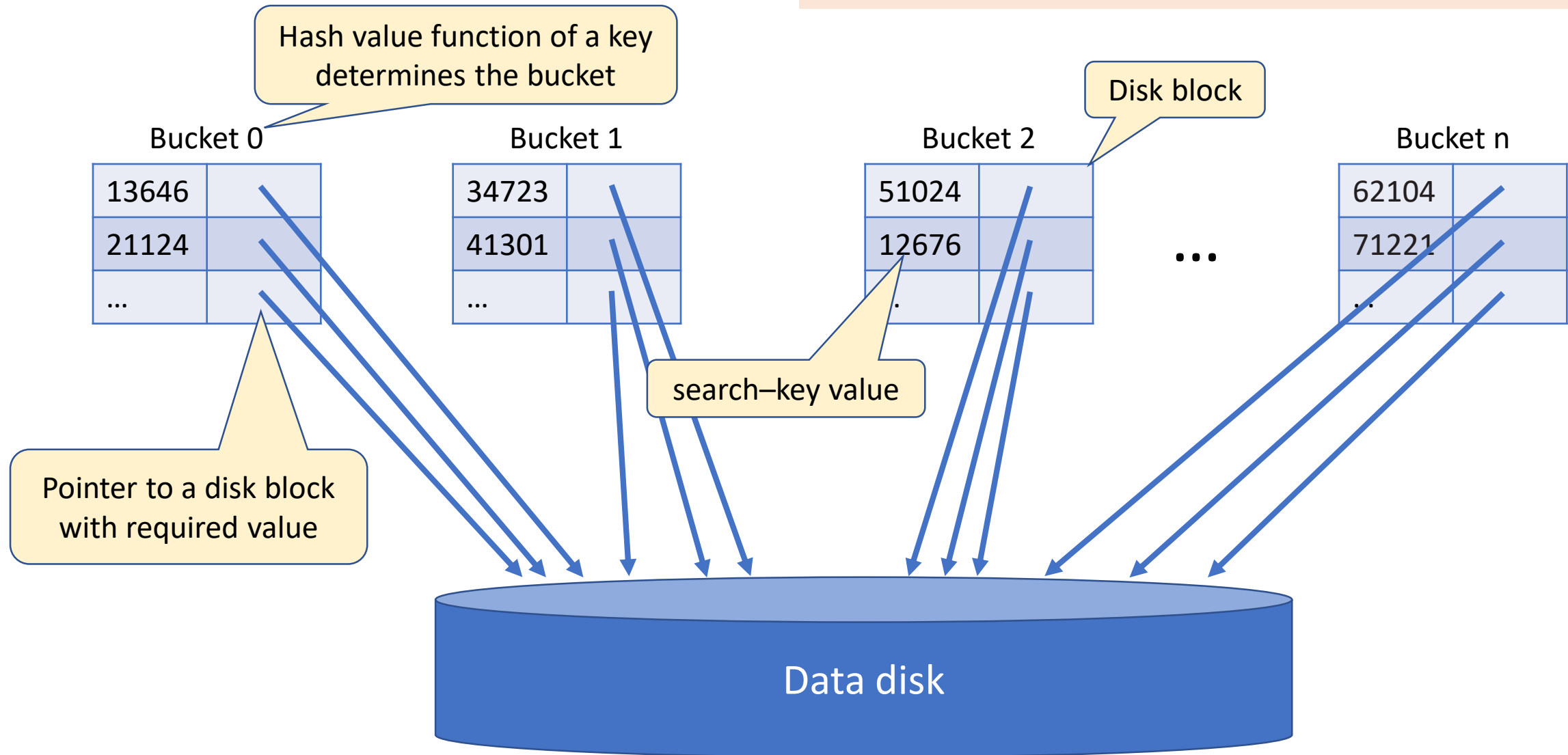
B⁺ tree index cost model for search

- We won't go into specifics of B⁺ tree insertion algorithm, but has several attractive qualities:
 - ~ **Same cost as exact search**
 - ***Self-balancing***: B⁺ Tree remains **balanced** (with respect to height) even after insert

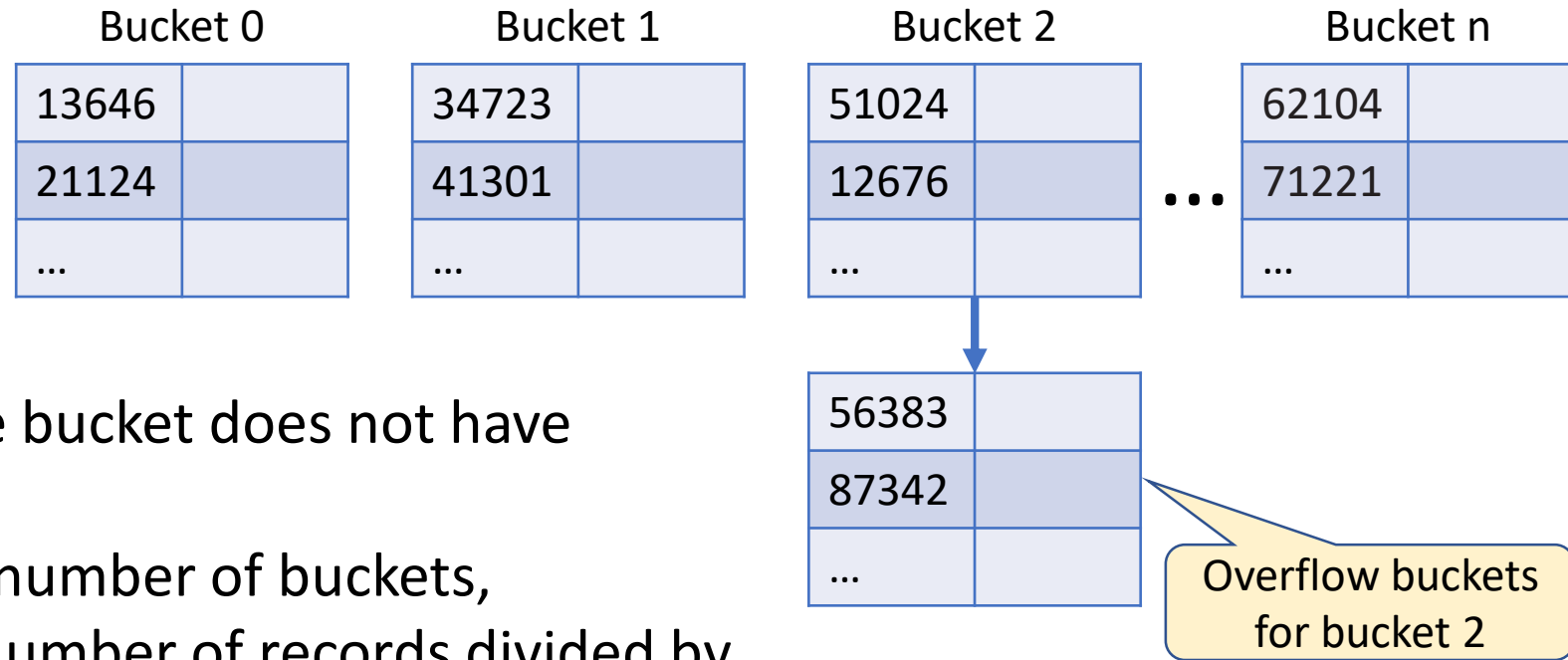
B⁺ Trees also (relatively) fast for single insertions!
*However, can become bottleneck if many insertions
(if fill-factor slack is used up...)*

Hash index

Hash indexes are often stored in RAM
(in-memory)



Hash index



Bucket is overflow when the bucket does not have enough space:

- **Insufficient buckets.** The number of buckets, must be higher than the number of records divided by the number of records that will fit in a bucket.
- **Skew.** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. Skew can occur for two reasons:
 1. Multiple records may have the same search key.
 2. The chosen hash function may result in non uniform distribution of search keys.

Hash index

- When the number of buckets is fixed when the index is created, is called **static hashing**. One of the problems with static hashing is that we need to know how many records are going to be stored in the index.
- A **dynamic hash** index can be rebuilt with an increased number of buckets. Rebuilding the index has the drawback that it can take a long time if the relations are large, causing disruption of normal processing.
- Hash function should assure that assigns search–key values distribution to buckets is **uniform** and **random**.
- Example of hash function for string of length n :

$$f(s(n)) = s[0] \cdot 31^{(n-1)} + s[1] \cdot 31^{(n-2)} + \dots + s[n-1]$$

A primary number

Bitmap index

- Bitmap indices provide a very compact representation for indexing attributes with **very few distinct values**. Intersection operations are extremely fast on bitmaps, making them ideal for supporting queries on multiple attributes.
- Each bitmap has as many bits as the number of records in the relation.

SID	Gender	CID	grade
12	F	CS101	NULL
12	F	CS900	5.0
55	M	CS900	NULL
34	M	CS102	3.5

Gender	Bitmaps
F	1100
M	0011

CID	Bitmaps
CS101	1000
CS900	0110
CS102	0001

4 bits because the table has 4 records

- Find female students attending course CS900: **1100 · 0110 = 0100**

Bitmap index

- Important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis. We can get the desired result from the bitmap index, without even accessing the relation.
- Bitmap indices are generally quite small compared to the actual relation size. The space occupied by a single bitmap is usually less than 1% of the space occupied by the relation.

Guidelines 1/3

- Large numbers of indexes on a table affect the performance of INSERT, UPDATE, DELETE statements because all indexes must be adjusted appropriately as data in the table changes.
- Avoid over-indexing heavily updated tables and keep indexes narrow, that is, with as few columns as possible.
- Use many indexes to improve query performance on tables with low update requirements, but large volumes of data.
- Indexing small tables may not be optimal because it can take the query optimizer longer to traverse the index searching for data than to perform a simple table scan.

Guidelines 2/3

- Indexes on views can provide significant performance gains when the view contains aggregations, table joins, or a combination of aggregations and joins.
- Create nonclustered indexes on the columns that are frequently used in predicates and join conditions in queries.
- Adding too many index columns can adversely affect disk space and index maintenance performance.
- Write queries that insert or modify as many rows as possible in a single statement, instead of using multiple queries to update the same rows. By using only one statement, optimized index maintenance could be exploited.

Guidelines 3/3

- Consider the cost versus benefit when adding new indexes. It may be preferable to consolidate additional query needs into an existing index. For example, consider adding one or two extra leaf level columns to an existing index, if it allows coverage of several critical queries, instead of having one exactly covering index per each critical query.

Thank you for your
attention!

Time for questions